OS Processes & Linux Filters

Study Guide covering Lecture 4 (Processes) and Section .

PART 1: PROCESS MANAGEMENT (LECTURE 4)

1. Process Concept & States

A process is a program in execution. As a process executes, it changes **state**. The state is defined by the current activity of that process.

The 5 Process States:

1. New: The process is being created.

2. Ready: The process is waiting to be assigned to a processor (in the Ready Queue).

3. Running: Instructions are being executed by the CPU.

4. Waiting: The process is waiting for some event to occur (e.g., waiting for I/O like typing on a keyboard).

5. Terminated: The process has finished execution.

💡 توضيح بالعربي: دورة حياة الـ Process

أي برنامج بتفتحه بيمر بـ 5 مراحل:

أول ما تدوس عليه، بيكون في مرحلة الـ **(New)** بيتجمع في الرامات.

أول ما يجهز، بيقف في طابور الـ **(Ready)** مستني دوره يدخل للبروسيسور.

لما البروسيسور يبدأ ينفذ الكود بتاعه، كدة هو **(Running)**.

لو البرنامج طلب يقرأ ملف من الهارديسك (عملية بطيئة)، البروسيسور هيقوله "أركن إنت على جمب في

مرحلة الـ **(Waiting)** عشان معطلكش"، ولما يخلص، بيرجع تاني لطابور الـ **(Ready)**.
ولما تقفل البرنامج، بيبقى **(Terminated)**.

2. PCB & Context Switch

Process Control Block (PCB)

Each process is represented in the OS by a PCB (also called a task control block). It contains all the info associated with a specific process:

- **Process state:** running, waiting, etc.
- **Program counter:** location of the *next* instruction to execute.
- **CPU registers:** contents of all process-centric registers.
- **CPU scheduling info:** priorities, scheduling queue pointers.
- **Memory-management info:** memory allocated to the process.
- **Accounting info:** CPU used, clock time elapsed since start.
- **I/O status info:** I/O devices allocated, list of open files.

Context Switch

When the CPU switches to another process, the system must **save the state** of the old process (into its PCB) and **load the saved state** (from the PCB) for the new process.

Context Switch Overhead:
Context-switch time is pure overhead; **the system does no useful work while switching.** The more complex the OS and the PCB, the longer the context switch takes.

💡 توضيح بالعربي: الـ PCB والـ Context Switch

الـ PCB: هو الـ "CV" أو الملف الشخصي بتاع كل Process عند الـ OS. مكتوب فيه كل تفصيلة عنها (حالتها، واصلة لفين في الكود، فاتحة ملفات إيه).

الـ Context Switch: تخيل إنك بتذاكر OS (ده Process A)، فجأة جالك تليفون. هتعمل إيه؟ هتعلم إنت

وقفت فين في الكتاب، وتشيله (Saving State). وترد على التليفون (Loading State لـ Process B). الوقت اللي ضاع في "تطليع وتدخيل الكتب" ده مفيش فيه أي إنجاز، وده اللي بنسميه **Overhead**.

3. Process Scheduling

The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization. The **process scheduler** selects an available process for program execution on the CPU.

Scheduling Queues

- **Job Queue:** Set of all processes in the system.
- **Ready Queue:** Set of all processes residing in main memory, ready and waiting to execute.
- **Device (Wait) Queues:** Set of processes waiting for a specific I/O device.

Types of Schedulers

Short-term Scheduler (CPU Scheduler):
Selects which process should be executed next and allocates CPU. Invoked very frequently (milliseconds), must be very fast.

Long-term Scheduler (Job Scheduler):
Selects which processes should be brought into the ready queue from disk. Invoked infrequently, controls the **degree of multiprogramming**.

Medium-term Scheduler:
Removes processes from memory temporarily (Swapping) to reduce the degree of multiprogramming, then brings them back later.

Process Categories

- **I/O-bound process:** Spends more time doing I/O than computations. (Many short CPU bursts).
- **CPU-bound process:** Spends more time doing computations. (Few very long CPU bursts).
- *Note:* A good long-term scheduler selects a good mix of both to keep both the CPU and I/O devices busy.

💡 توضيح بالعربي: الطوابير والمُجدولات

عشان الـ CPU ميفضلش فاضي، عندنا طوابير (Queues) للبرامج.

اللي بيتحكم في الطوابير دي هما الـ **Schedulers**:

1. **Long-term:** ده البواب اللي بيقرر مين يدخل من الهارديسك للرامات (الـ Ready Queue). ده بيتحكم في الزحمة (Degree of multiprogramming). لازم يدخل برامج متوازنة (شوية بيسحبوا CPU زي الألعاب، وشوية بيسحبوا I/O زي تحميل الملفات).
2. **Short-term:** ده اللي جوه الرامات وبيختار مين من الطابور يدخل للـ CPU دلوقتي حالاً. ده لازم يكون سريع جداً جداً لأنه بيشتغل كل جزء من الثانية.
3. **Medium-term:** لو الرامات اتزحمت أوي، ده بيطرد بعض البرامج للهارديسك مؤقتاً (Swapping) عشان يخفف الحمل.

4. Process Creation & Termination

Process Creation

A Parent process creates children processes, forming a **tree of processes**. Processes are identified and managed via a process identifier **(PID)**.

- In UNIX/Linux, the `fork()` system call creates a new process (the child). The child is an exact duplicate of the parent.
- The `exec()` system call is used *after* a fork to replace the process' memory space with a brand new program.

Process Termination

- Process executes last statement and asks OS to delete it using the `exit()` system call.
- Output data from child is passed to the parent (via `wait()`).
- Process' resources (memory, open files, buffers) are deallocated by the OS.

5. Zombies, Orphans & Chrome Arch

When a child terminates, its resources are freed, but its PCB remains until the parent acknowledges its death by calling `wait()`.

🧟 Zombie Process

If a child terminates, but the parent **did not invoke `wait()`**, the child process is a zombie. All processes exist as zombies briefly until `wait()` is called.

😢 Orphan Process

If a parent terminates **without invoking `wait()`**, its children become orphans. The OS assigns the `init` (root) process as the new parent to clean them up.

Multiprocess Architecture (Chrome Browser)

Many old browsers ran as a single process. If one website crashed, the whole browser crashed. Google Chrome uses a **multiprocess** architecture:

- **Browser process:** Manages User Interface, disk, and network I/O.
- **Renderer process:** Renders web pages (HTML, Javascript). *A new renderer is created for each website opened (Each tab is a separate process).*
- **Plug-in process:** For each type of plug-in.

💡 توضيح بالعربي: إيه حكاية الـ Zombies والـ Orphans؟

في لينكس، لما الـ (Child) يخلص شغله ويموت، الـ OS بيفضي راماته، بس بيسيب اسمه في السجلات لحد ما الـ (Parent) يقرأ رسالة الوفاة (بدالة `wait()`).

الـ Zombie: هو Child خلص، بس أبوه مشغول وماعملش `wait()` عشان يستلم جثته! فبيفضل اسمه متعلق كزومبي.

الـ Orphan: ده Child شغال، بس أبوه مات قبل ما يخلص! الـ OS بيبعته لـ "ملجأ" (أول Process في النظام اللي اسمها `init`) وهي بتتبناه وبتعمله `wait()` لما يخلص.

متصفح كروم: ليه كروم بيسحب رامات كتير؟ لأنه بيعمل Process كاملة (Renderer) لكل Tab بتفتحها. الميزة العظيمة هنا إن لو موقع هنج، التاب دي بس اللي هتقفل وباقي المتصفح هيفضل شغال زي الفل!

6. Interprocess Communication (IPC)

Processes within a system may be independent or cooperating. Cooperating processes can affect or be affected by other processes, and they need **IPC** to exchange data.

1. Shared Memory

A region of memory that is shared by cooperating processes is established. Processes can then exchange information by reading and writing data to this shared region.

Pros: Very fast once established.

2. Message Passing

Communication takes place by means of messages exchanged between the cooperating processes via the OS (using `send()` and `receive()`).

Pros: Easier to implement, good for smaller amounts of data.

💡 توضيح بالعربي: البرامج بتكلم بعض إزاي؟

عشان برنامجين يتبادلوا داتا مع بعض (IPC)، عندهم طريقين:

1. Shared Memory: الـ OS بيحجز ليهم حتة في الرامات مشتركة بينهم. كأنهم قاعدين على نفس الترابيزة وبيقرأوا ويكتبوا في نفس الورقة. دي طريقة سريعة جداً بس محتاجة تنظيم عشان ميكتبوش فوق كلام بعض.

2. Message Passing: كأنهم بيبعتوا رسائل لبعض عن طريق البريد (الـ OS). دي طريقة أبطأ شوية بس أسهل في التنظيم ومفيدة لو البرنامجين على أجهزة مختلفة (زي الشات).

PART 2: LINUX ADVANCED (SECTION)

7. Users & Services Overview

Linux User Types Review

- **Root user (ID: 0):** Super admin, full permissions.
- **Regular user (ID >= 1000):** Standard user for daily tasks.
- **Service account (0 < ID < 1000):** Accounts used by software packages to run safely without a home directory.

What is a Service in Linux?

A service in Linux is a program (or set of programs) that runs in the background, providing various functions to the system. Services are started automatically when the system boots up and continue running until the system is shut down (e.g., Network manager, Web server).

8. File Permissions in Linux

Every file and directory in Linux has three sets of permissions: User (owner), Group, and Others.

```
Example output of ls -l:
-rwxrw-r-- 1 salma salma 4096 file.txt
```

- **Type:** - (File), d (Directory).
- **Owner (u):** The user who created the file (rwx).
- **Group (g):** Users in the file's group (rw-).
- **Others (o):** Everyone else (r--).

Numeric Mode (Absolute Mode)

Read (r) = 4	+	Write (w) = 2	+	Execute (x) = 1	=	Total (0 to 7)

```
chmod 755 file.txt          Owner(7=rwx), Group(5=r-x), Others(5=r-x).
```

```
chmod u+x file.txt          Symbolic mode: Add execute to User.
```

```
chown user file.txt         Change the owner of the file.
```

```
chgrp group file.txt        Change the group of the file.
```

9. I/O Redirection

In Linux, everything is a file. Commands read from standard input and write to standard output. We can redirect these streams.

- **stdin (0):** Standard Input (default: Keyboard).
- **stdout (1):** Standard Output (default: Screen).
- **stderr (2):** Standard Error (default: Screen).

```
ls > files.txt                    Redirect stdout to a file (Overwrites file).
```

```
date >> files.txt                 Redirect stdout to a file (Appends to end of file).
```

```
cat < files.txt                   Redirect stdin from a file instead of keyboard.
```

```
ls /fake_dir 2> errors.txt        Redirect only standard errors to a file.
```

```
ls /fake_dir &> all.txt           Redirect BOTH stdout and stderr to the same file.
```

10. Piping (|)

Piping connects the Standard Output (stdout) of one command directly to the Standard Input (stdin) of another command.

```
Command 1 | Command 2
```

```
ls -l /etc | more     Take huge output of ls, pass it to 'more' to view page by page.
```

Command Substitution $( )

Allows the output of a command to replace the command name itself.

```
echo "Today is $(date)"
```
The $(date) will be executed first, and its result is placed inside the string.

> 💡 **توضيح بالعربي: الفرق بين الـ Redirection و الـ Pipe**
>
> **الـ Redirection (>):** بياخد ناتج أمر ويرميه جوه **ملف** يحفظه. (أمر ← ملف).
>
> **الـ Pipe (|):** بياخد ناتج أمر، وبدل ما يطبعه على الشاشة، بيبعته كمدخل **لأمر تاني** يشتغل عليه. (أمر ← أمر).
>
> يعني لو عايز تطلع كل الملفات اللي في النظام، وتدور فيهم على كلمة معينة، بتستخدم البايب: `ls | grep "word"`.

11. Text Filters & Useful Commands

1. wc (Word Count)

```
wc -l file.txt
```
Count lines.

```
wc -w file.txt
```
Count words.

```
wc -c file.txt
```
Count characters/bytes.

2. sort (Sorting Data)

```
sort file.txt
```
Sort alphabetically.

```
sort -n file.txt
```
Sort numerically.

```
sort -r file.txt
```
Reverse sort.

```
sort -t: -k3 -n /etc/passwd    Advanced: Split by ':' (-t:), sort by 3rd column (-k3) numerically (-n).
```

3. cut (Extracting Columns)

```
cut -d: -f1,7 /etc/passwd
```

Uses ':' as delimiter (-d:), and extracts Field 1 and Field 7 (-f1,7).

4. grep (Global Regular Expression Print)

Searches for a specific pattern within a file or stream.

```
grep "word" file.txt    Find lines containing 'word'.
```

```
grep -i "word" file.txt    Case-insensitive search.
```

```
grep -v "word" file.txt    Invert match (show lines DO NOT contain 'word').
```

```
grep "^root" /etc/passwd    Lines that START with 'root'.
```

```
grep "bash$" /etc/passwd    Lines that END with 'bash'.
```

5. Other Useful Commands (from tasks)

```
nautilus    Opens the GUI File Manager (Files) for the current directory from terminal.
```

```
compgen -c > /tmp/commands.list
```

The compgen -c command lists all available user commands in Linux. Here we redirect its output to a file.